Final Project Report

AI-Powered Document Intelligence Assistant for Public Sector

1. Introduction

Public offices work with many long documents such as budget reports, policies and official letters. Some are scanned and cannot be searched. This project is a web-based AI assistant that helps users understand such documents quickly. The user uploads a PDF, asks questions, and gets answers with the source file and page number. The assistant can also create a summary, key points and action items.

2. Problem Statement and Objective

Problem: Finding one fact in a long PDF takes time, and scanned PDFs cannot be searched at all. AI tools can also give answers that are not in the document, which is risky for official work.

Objective: Build a practical assistant that reads uploaded PDFs (including scanned ones), answers only from the document, shows its sources, and says clearly when the answer is not available.

3. Requirements and How They Are Met

Requirement	How the project meets it
PDF upload	POST /documents/upload and the Upload tab in the Gradio interface
Text extraction and OCR	PyMuPDF reads native text. Pages with fewer than 30 characters are rendered as images and read with Tesseract OCR
Summarization	POST /documents/summary
Question answering	POST /chat/ask using retrieval-augmented generation (RAG)

Source references	Each answer returns the file name and page. Citations written by the AI are checked against the saved page numbers
Key point extraction	POST /documents/key-points (6 to 10 points)
Action item generation	POST /documents/action-items
Fallback response	If no relevant text is found, or the model reports it is not in the document, the app returns a fixed fallback message
Simple interface	Gradio app with three tabs: Upload, Ask a question, Summary and insights
Error handling and validation	File type, empty file and size checks; clear messages for missing OCR, AI quota (429) and AI busy (503)
Configurable settings	pydantic-settings reads the model name, chunk size, top-k, similarity threshold and limits from environment variables
Documentation	README with installation, configuration and run steps, plus this report

4. Tools and Technologies

- **Python 3.12** – main language
- **FastAPI and Uvicorn** – backend API
- **PyMuPDF** – native PDF text extraction and page rendering
- **Tesseract OCR (pytesseract) and Pillow** – OCR for scanned pages
- **sentence-transformers (all-MiniLM-L6-v2)** – text embeddings
- **ChromaDB** – local vector database (cosine similarity)
- **Google Gemini (google-genai)** – answer, summary, key point and action item generation
- **Gradio** – user interface
- **Pydantic and pydantic-settings** – data models and configuration
- **pytest and test scripts** – testing

5. System Architecture

The system has a Gradio interface that calls a FastAPI backend over HTTP.

When a document is uploaded:

1. The file is checked (PDF only, not empty, within the size limit) and saved with a unique name.
2. Text is extracted page by page. OCR runs only on pages with very little text. The longer result is kept.
3. Text is cleaned and split into chunks of 800 characters with 150 characters of overlap. Each chunk keeps its page number.
4. Embeddings are created and stored in ChromaDB with the document ID, file name and page.
5. Older documents are removed, so only one document is active.

6. Implementation Highlights

Smart OCR. Running OCR on every page is slow. The app first uses native text and only runs OCR on weak pages, so normal PDFs stay fast and mixed PDFs still work. The result is reported as native, OCR or mixed.

Grounded answers. The prompt allows only the supplied context, forbids invented facts and treats the document as information, not instructions. A relevance filter removes weak matches before the model is called.

Checked citations. The model cites sources like (Source 2, Page 18). The app reads these citations and keeps a source only if that page really matches the saved metadata. Wrong citations are dropped.

Document isolation. Every search is filtered by document_id, and a new upload replaces the old index only after the new chunks are stored successfully. This prevents answers from mixing two documents.

Reliable AI calls. Temporary Gemini server errors are retried up to 3 times. Quota errors return a clear 429 message. Other failures return a safe error without exposing internal details.

Configuration. All important settings can be changed through environment variables, and the API key is never stored in code.

8. Challenges and Solutions

Challenge	Solution
Scanned and mixed PDFs	Native text first, OCR only on weak pages
Questions about specific figures were not always retrieved	Tuned chunk size, top-k and similarity threshold, and tested with real report data
The AI can write wrong page numbers	Validate each citation against saved page metadata
Old and new documents could mix	Use a document ID filter and replace the old index after a successful upload
Gemini quota limits and busy servers	Retry logic and clear user-facing messages

9. Limitations

- Only one document is active at a time, and there is no user login.
- OCR is set to English only.
- Only PDF files are supported.
- The AI answer quality depends on the Gemini model and the quality of the source text or scan.

10. Future Improvements

- Support many documents per user, with login
- Add Urdu OCR and multi-language questions
- Summarize very long documents part by part
- Add automated tests with assertions and a CI pipeline
- Provide Docker setup and a hosted demo
- Let users export summaries and action items

11. Conclusion

The project delivers a working assistant that reads normal and scanned PDFs, answers from the document with checked sources, and clearly refuses when the answer is missing. It meets the core requirements of the final assignment and shows the skills built during the internship: backend development, RAG, OCR, prompt design, error handling and documentation.